

文章笔记

成为 Claude 架构师的完整学习路径

基于公开课程资料整理

2026 年 4 月 1 日

原文作者: hoeem

发布日期: 2026 年 3 月 15 日

来源平台: X (Twitter) Article

原文链接: <https://x.com/hoeem/status/2033198345045336559>

项目主页:

文章背景

本文基于 Anthropic 官方 Claude Certified Architect 认证考试指南，由作者 hoeem 拆解整理而成。考试本身仅面向 Claude 合作伙伴开放，但其考纲覆盖的知识体系——Claude Code、Agent SDK、API 和 MCP——是构建生产级 AI 应用的核心技能栈。本笔记对原文五大领域进行系统化整理与补充。

目录

1 概述：Claude 架构师知识体系	3
1.1 本章小结	3
2 领域一：Agentic 架构与编排 (27%)	3
2.1 三大反模式	4
2.2 子 Agent 的上下文隔离	4
2.3 安全关键场景的 Hook 机制	4
2.4 推荐实践项目	5
2.5 本章小结	5
3 领域二：工具设计与 MCP 集成 (18%)	5
3.1 工具描述是工具选择的核心	5
3.2 tool_choice 参数详解	6
3.3 工具数量的上限原则	6
3.4 推荐实践项目	6
3.5 本章小结	6
4 领域三：Claude Code 配置与工作流 (20%)	6
4.1 CLAUDE.md 三级层次结构	6
4.2 路径特定规则 (Path-Specific Rules)	7
4.3 Plan Mode vs Direct Execution	7
4.4 其他关键概念	7
4.5 推荐实践项目	7
4.6 本章小结	8
5 领域四：Prompt 工程与结构化输出 (20%)	8
5.1 模糊指令 vs 明确指令	8
5.2 Few-shot 示例的杠杆效应	8
5.3 tool_use 与 JSON Schema	8
5.4 Message Batches API	9
5.5 推荐实践项目	9
5.6 本章小结	9
6 领域五：上下文管理与可靠性 (15%)	9
6.1 渐进式摘要的陷阱	9
6.2 "Lost in the Middle" 效应	10
6.3 升级触发器 (Escalation Triggers)	10
6.4 错误传播的正确方式	10
6.5 推荐实践项目	11
6.6 本章小结	11

7 总结与延伸	11
7.1 五大领域核心要点回顾	11
7.2 Anthropic 官方推荐学习资源	11
7.3 学习建议	12

1 概述：Claude 架构师知识体系

Claude Certified Architect 认证考试涵盖五大领域，考察候选人构建生产级 Claude 应用的综合能力。虽然考试仅对合作伙伴开放，但掌握这些知识本身就足以让你构建高质量的 AI 应用。

考试覆盖的四大核心技术

1. **Claude Code**: Anthropic 官方 CLI 工具，支持代码生成、CI/CD 集成
2. **Claude Agent SDK**: 用于构建 agentic 应用的开发套件
3. **Claude API**: 模型调用接口，含 `tool_use`、Batches API 等
4. **Model Context Protocol (MCP)**: 工具与上下文集成的标准协议

考试要求候选人能够完成以下六类典型项目：

- **客户支持解决 Agent**: Agent SDK + MCP + 升级机制 (escalation)
- **Claude Code 代码生成**: CLAUDE.md 配置 + plan mode + slash commands
- **多 Agent 研究系统**: coordinator-subagent 编排模式
- **开发者生产力工具**: 内置工具 + MCP servers
- **Claude Code CI/CD**: 非交互式管道 + 结构化输出
- **结构化数据抽取**: JSON schemas + `tool_use` + 验证循环

五大领域的权重分布如下表所示：

领域	权重	核心主题
Agentic Architecture & Orchestration	27%	Agent 循环、子 Agent 编排
Tool Design & MCP Integration	18%	工具描述、MCP 协议
Claude Code Configuration & Workflows	20%	CLAUDE.md 层级、规则
Prompt Engineering & Structured Output	20%	Few-shot、 <code>tool_use</code>
Context Management & Reliability	15%	上下文管理、错误传播

表 1: Claude Architect 考试五大领域及权重

1.1 本章小结

Claude 架构师需要同时掌握 Claude Code、Agent SDK、API 和 MCP 四大技术栈。考试通过六类典型项目来检验综合能力，其中 Agentic Architecture 占比最高（27%），是核心中的核心。

2 领域一：Agentic 架构与编排（27%）

这是权重最高的领域，考察候选人设计和实现 agent 系统的能力，包括 agentic loop 机制、子 Agent 编排以及安全关键场景下的 hook 设计。

2.1 三大反模式

考试会直接测试你能否识别以下三个常见的反模式：

必须立即拒绝的三种 Agent 停止策略

1. **解析自然语言判断循环终止**：通过分析模型回复的文本内容来决定是否停止循环。这种方式不可靠，因为自然语言具有歧义性。
2. **任意迭代次数上限作为主要停止机制**：例如“最多运行 10 轮”。这不是一个语义上合理的终止条件，应基于 `stop_reason` 来判断。
3. **检查 assistant 文本作为完成指示器**：模型输出文本并不意味着任务完成，正确做法是检查 API 返回的 `stop_reason` 字段。

2.2 子 Agent 的上下文隔离

子 Agent 不共享 Coordinator 的内存

这是原文作者强调的“最大错误”：很多人假设子 Agent (subagent) 可以访问 coordinator 的上下文。实际上，子 Agent 的上下文是完全隔离的，所有信息必须通过 prompt 显式传递。

这意味着在设计多 Agent 系统时，你需要：

- 明确定义 coordinator 向每个 subagent 传递的信息边界
- 在 subagent 的 prompt 中包含所有必要的上下文
- 设计清晰的返回值结构，确保 coordinator 能整合各 subagent 的结果

2.3 安全关键场景的 Hook 机制

高风险场景必须用程序化 Hook 而非 Prompt 指令

当涉及金融交易或安全敏感操作时，仅靠 prompt 中的指令是不够的。必须通过 **hook** 和 **prerequisite gate** 以编程方式强制执行工具调用顺序。

例如，在一个涉及退款操作的 Agent 中：

- 使用 `PostToolUse` hook 规范化数据格式
- 使用工具调用拦截 hook 阻止违反策略的操作
- 通过 `prerequisite gate` 确保在执行退款前必须先完成身份验证

2.4 推荐实践项目

动手练习：多工具 Agent

构建一个包含 3-4 个 MCP 工具的 Agent，实现以下功能：

- 正确的 `stop_reason` 处理
- 一个 `PostToolUse` hook 用于规范化数据格式
- 一个工具调用拦截 hook 用于阻止策略违规

这一个练习就能覆盖领域一的大部分考点。

2.5 本章小结

Agentic 架构的核心在于三点：正确使用 `stop_reason` 而非自然语言解析来控制 Agent 循环；理解子 Agent 上下文隔离的本质并显式传递信息；在高风险场景中用程序化 hook 而非 prompt 指令来保障安全。

3 领域二：工具设计与 MCP 集成 (18%)

本领域聚焦于如何设计高质量的工具描述 (tool description)、理解 MCP 协议，以及正确配置 `tool_choice` 参数。

3.1 工具描述是工具选择的核心

工具描述决定 Claude 的工具选择行为

Tool description 是 Claude 用于决定调用哪个工具的主要依据。如果描述模糊或多个工具描述重叠，工具选择将变得不可靠。

原文举了一个典型案例：`get_customer` 和 `lookup_order` 两个工具的描述过于相似，导致 Claude 经常选错工具。面对这种情况，正确的修复方案不是：

- 添加 few-shot 示例（治标不治本）
- 构建路由分类器（过度工程）
- 合并工具（丧失功能边界）

正确做法：重写工具描述，使每个工具的职责边界清晰、不重叠。

3.2 tool_choice 参数详解

选项	行为
"auto"	模型自行决定是调用工具还是直接回复文本
"any"	模型 必须 调用某个工具，但由模型选择具体调用哪一个
forced selection	模型 必须 调用指定的特定工具

表 2: tool_choice 三种模式对比

3.3 工具数量的上限原则

单个 Agent 的工具不应超过 4-5 个

给一个 Agent 提供 18 个工具会严重降低工具选择的准确率。正确做法是将工具按角色分配给不同的子 Agent，每个子 Agent 只携带 4-5 个与其职责相关的工具。

3.4 推荐实践项目

动手练习：工具描述对比实验

创建两个功能相似的 MCP 工具，故意写模糊的描述使其产生路由错误。然后修复描述，亲身体验描述质量对工具选择的影响。

3.5 本章小结

工具设计的关键在于：写出清晰、无歧义的工具描述；熟练掌握 tool_choice 的三种模式及适用场景；控制单个 Agent 的工具数量在 4-5 个以内，通过子 Agent 分工来扩展能力。

4 领域三：Claude Code 配置与 workflow (20%)

这个领域考察的是你是否不仅会使用 Claude Code, 还能团队配置它。核心概念包括 CLAUDE.md 层级体系、规则目录、plan mode 与 direct execution 的选择。

4.1 CLAUDE.md 三级层次结构

CLAUDE.md 的三个层级

1. **User-level**: ~/.claude/CLAUDE.md —— 个人配置，不受版本控制，不会与团队共享
2. **Project-level**: .claude/CLAUDE.md (或项目根目录的 CLAUDE.md) —— 项目级配置，随代码库版本控制
3. **Directory-level**: 子目录中的 CLAUDE.md —— 仅对该目录及其子目录生效

考试常见陷阱：User-level 配置不共享

如果团队成员缺少某些指令，很可能是因为这些指令被放在了 user-level 的 CLAUDE.md 中。由于该文件不受版本控制，其他团队成员不会看到这些配置。团队共享的配置必须放在 project-level。

4.2 路径特定规则 (Path-Specific Rules)

.claude/rules/ 目录的 Glob 模式

.claude/rules/ 目录支持 YAML frontmatter 中的 glob 模式，例如 `**/*.test.tsx` 可以将特定约定应用于整个代码库中匹配的文件。这是 directory-level 的 CLAUDE.md 做不到的，因为后者只对所在目录及子目录生效，无法跨目录匹配文件路径模式。

4.3 Plan Mode vs Direct Execution

模式	适用场景
Plan Mode	单体应用重构、多文件迁移、架构决策等需要全局视角的任务
Direct Execution	单文件 bug 修复、单个验证检查等范围明确的小任务

表 3: Plan Mode 与 Direct Execution 的适用场景

4.4 其他关键概念

需要掌握的其他 Claude Code 概念：

- **context: fork**：在 skill frontmatter 中使用，隔离冗长输出，避免污染主会话上下文
- **-p flag**：非交互式模式，用于 CI/CD 管道
- **独立审查实例**：用一个新的 Claude Code 实例来审查代码，比在同一个 session 中自我审查效果更好

4.5 推荐实践项目

动手练习：完整 Claude Code 配置

创建一个项目，包含：

- 完整的 CLAUDE.md 三级层次结构
- .claude/rules/ 中带 glob 模式的规则文件
- 一个使用 context: fork 的 skill
- .mcp.json 中配置带环境变量展开的 MCP server

然后分别用 plan mode 处理多文件重构，用 direct execution 处理单个 bug 修复，体会两者的区别。

4.6 本章小结

Claude Code 的团队配置能力是架构师的分水岭。核心要点：理解 CLAUDE.md 的三级层次并将共享配置放在 project-level；利用 `.claude/rules/` 的 glob 模式实现跨目录的路径特定规则；根据任务范围选择 plan mode 或 direct execution。

5 领域四：Prompt 工程与结构化输出（20%）

这个领域的核心原则只有两个字：**明确**（be explicit）。

5.1 模糊指令 vs 明确指令

模糊指令无法提升精度

以下常见指令实际上不起作用：

- “Be conservative”——不会真正提升精度
- “Only report high-confidence findings”——不会真正减少误报

有效做法：精确定义哪些问题应该上报、哪些应该跳过，并为每个严重级别提供具体的代码示例。

5.2 Few-shot 示例的杠杆效应

Few-shot 是考试中测试的最高杠杆技术

使用 2-4 个精心设计的示例，展示以下内容：

- 模糊情况的处理方式
- 为什么选择了某个动作而非其他替代方案
- 推理过程的展示

关键不在于示例数量，而在于选择**模糊边界案例**（ambiguous cases）作为示例。

5.3 tool_use 与 JSON Schema

tool_use 配合 JSON schema 可以消除**语法错误**，但**无法消除语义错误**。在 schema 设计中需要注意：

- **nullable 字段**：当源数据可能缺失时，使用 nullable 字段而非让模型编造值
- **“unclear” 枚举值**：为模型提供一个“不确定”的选项
- **“other” + detail 字符串**：覆盖预定义枚举之外的情况

5.4 Message Batches API

Batches API 的特性与适用场景

- 节省 50% 成本
- 处理时间最长 24 小时，无延迟 SLA
- 不支持多轮 tool calling
- **适用**：隔夜批量报告生成
- **不适用**：需要阻塞等待结果的 pre-merge 检查（应使用同步 API）

5.5 推荐实践项目

动手练习：结构化数据抽取管道

构建一个使用 `tool_use` 的数据抽取管道：

- 定义包含 `required`、`optional` 和 `nullable` 字段的 JSON schema
- 实现 `validation-retry` 循环
- 通过 Batches API 批量处理
- 按 `custom_id` 处理失败情况

5.6 本章小结

Prompt 工程的核心是“明确”：用具体标准替代模糊指令，用 2-4 个边界案例的 few-shot 示例来引导模型行为。结构化输出方面，`tool_use` + JSON schema 解决语法问题，`nullable` 字段和“unclear”枚举值解决语义问题。Batches API 适合非实时批量任务。

6 领域五：上下文管理与可靠性（15%）

权重最小的领域，但原文强调“这里的错误会级联到所有其他地方”。

6.1 渐进式摘要的陷阱

渐进式摘要会丢失关键事务数据

Progressive summarisation（渐进式摘要）会丢失金额、日期、订单号等事务性数据。正确做法是维护一个持久的“**case facts**”块，包含所有提取的关键数据字段，**永不摘要**，并在每次 prompt 中包含。

6.2 “Lost in the Middle” 效应

关键信息放在开头

模型容易遗漏埋在长输入中间的内容 (“lost in the middle” 效应)。解决方案：将关键摘要和重要发现放在输入的开头位置。

6.3 升级触发器 (Escalation Triggers)

触发器	描述	可靠性
客户主动请求人工	立即执行，不附加条件	可靠
策略空白	Agent 遇到规则未覆盖的情况	可靠
无法推进	Agent 无法继续完成任务	可靠
情绪分析	基于客户情绪判断是否升级	不可靠
自报置信度分数	模型自己报告的置信度	不可靠

表 4: 升级触发器的可靠性对比

考试陷阱：不可靠的升级触发器

情绪分析 (sentiment analysis) 和模型自报置信度分数 (self-reported confidence scores) 是考试中常见的干扰选项。它们看起来合理，但实际上不够可靠，不应作为升级触发条件。

6.4 错误传播的正确方式

正确的错误传播应包含结构化上下文：

- 失败类型 (failure type)
- 尝试的查询 (attempted query)
- 部分结果 (partial results)
- 替代方案 (alternatives)

两种必须避免的反模式：

- 静默吞掉错误 (silently suppressing errors)
- 因单个失败而终止整个工作流 (killing entire workflows on single failures)

6.5 推荐实践项目

动手练习：错误传播与部分结果

构建一个包含两个子 Agent 的 coordinator 系统：

- 模拟一个子 Agent 超时
- 验证 coordinator 能收到结构化的错误上下文
- 确认 coordinator 能基于部分结果继续推进
- 测试面对冲突信息源时的处理逻辑

6.6 本章小结

上下文管理的关键要点：用持久化的”case facts” 块替代渐进式摘要来保留事务数据；利用”lost in the middle” 效应的规律将关键信息前置；仅使用可靠的升级触发器（客户请求、策略空白、无法推进）；错误传播要结构化，既不静默吞掉也不因单点失败而全面崩溃。

7 总结与延伸

7.1 五大领域核心要点回顾

1. **Agentic 架构** (27%)：基于 `stop_reason` 的循环控制、子 Agent 上下文隔离、程序化 hook 保障安全
2. **工具设计与 MCP** (18%)：清晰的工具描述、合理的 `tool_choice` 配置、每个 Agent 4-5 个工具的上限
3. **Claude Code 配置** (20%)：三级 CLAUDE.md 层次、glob 模式规则、plan mode vs direct execution
4. **Prompt 工程** (20%)：明确指令、边界案例 few-shot、`tool_use` + JSON schema、Batches API
5. **上下文管理** (15%)：持久化 case facts、信息前置、可靠升级触发器、结构化错误传播

7.2 Anthropic 官方推荐学习资源

1. **Building with the Claude API**——API 基础与高级用法
2. **Introduction to Model Context Protocol**——MCP 协议入门
3. **Claude Code in Action**——Claude Code 实战演示
4. **Claude 101**——Claude 基础知识

7.3 学习建议

从实践项目入手

每个领域都配有一个推荐实践项目。建议按以下顺序完成：

1. 先完成领域一的多工具 Agent 项目（覆盖面最广）
2. 然后是领域三的 Claude Code 配置项目（直接提升日常效率）
3. 再完成领域二和四的工具描述与数据抽取项目
4. 最后完成领域五的错误传播项目（整合前四个领域的知识）

正如原文作者所言：你不需要证书来构建生产级应用，你需要的是知识本身。掌握这五大领域的核心概念并通过实践项目内化，你就具备了成为 Claude 架构师的能力。